

Reinforcement Learning i Transfer Learning aplicats al Moviment d'un Agent en un Món Virtual

Adrià Rodríguez

June 9, 2026

Contents

1	Introducció	3
1.1	Context	3
1.2	Motivació	3
1.3	Objectius	4
1.4	Metodologia	5
1.4.1	Estructura de la memòria	5
2	Fonaments Teòrics i Elements de RL	7
2.1	Conceptes generals del RL	8
2.2	Mètodes basats en la funció de valor	10
2.2.1	Relació entre els mètodes basats en valor vs els basats en política	10
2.2.2	Equació de Bellman	11
2.3	Framework de RL	12
2.3.1	Entorn	12
2.3.2	Algorisme	13
3	Primer contacte:	
	Entorn Discret	14
3.1	Implementació de la Graella	14
3.1.1	Estat	15
3.1.2	Acció	15
3.1.3	Estructura de dades	15
3.1.4	Pas	16
3.1.5	Reward	16
3.2	Primer algorisme: Q-Learning	16
3.2.1	Pseudocodi	18
3.2.2	Detalls de la implementació	19

3.2.3	Estudi de l'algorisme a l'entorn discret	19
4	Entorn continu	26
4.1	Implementació de l'entorn continu	26
4.1.1	Estat	26
4.1.2	Acció	27
4.1.3	Estructura de dades	27
4.1.4	Pas	28
4.1.5	Reward	28
4.2	Algorisme: Deep Q Learning	28
4.2.1	Pseudocodi i execució	30

1 Introducció

1.1 Context

La intel·ligència artificial ha experimentat un creixement extraordinari durant les darreres dècades, impulsada principalment pels avenços en aprenentatge automàtic i en la disponibilitat de grans volums de dades i capacitat computacional. Dins d'aquest àmbit, l'aprenentatge per reforç (Reinforcement Learning, RL) constitueix una branca especialment interessant, ja que permet entrenar agents capaços de prendre decisions de manera autònoma a partir de la seva interacció amb un entorn.

A diferència d'altres paradigmes com l'aprenentatge supervisat, on el model disposa d'exemples etiquetats durant l'entrenament, en l'aprenentatge per reforç l'agent ha d'aprendre mitjançant prova i error. A cada instant, l'agent observa l'estat de l'entorn, selecciona una acció i rep una recompensa que indica la qualitat de la decisió presa. L'objectiu final és aprendre una política de comportament que maximitzi la recompensa acumulada al llarg del temps.

Aquest paradigma ha demostrat ser eficaç en una gran varietat de problemes, incloent videojocs, optimització industrial, control robòtic i conducció autònoma.

1.2 Motivació

La principal motivació per escollir aquest tema com a Treball de Fi de Grau ha estat l'interès per aprofundir en un àmbit de la intel·ligència artificial diferent dels continguts treballats majoritàriament durant la carrera. La major part dels algorismes estudiats en les assignatures relacionades amb l'aprenentatge automàtic es basaven en paradigmes d'aprenentatge super-

visat, fet que converteix aquest treball en una oportunitat excel·lent per jugar amb un camp que tracta amb aprenentatge no supervisat, molt utilitzat en l'actualitat.

1.3 Objectius

L'objectiu principal d'aquest treball és comprendre en profunditat el funcionament dels principals algorismes d'aprenentatge per reforç mitjançant la seva implementació des de zero, així com estudiar-ne les limitacions i els avantatges en diferents tipus de problemes. A més d'entrenar un agent de conducció autònoma en una ciutat simulada, utilitzant tècniques com el Transfer Learning per reduir el temps d'entrenament.

De manera específica, es plantegen els següents objectius:

- Desenvolupar un framework que permeti definir entorns, agents i processos d'entrenament de forma general i reutilitzable.
- Implementar diferents famílies d'algorismes d'aprenentatge per reforç, incloent Q-Learning, Deep Q-Networks (DQN), Proximal Policy Optimization (PPO) i mètodes evolutius.
- Analitzar les diferències entre aquests algorismes en termes de convergència, capacitat de generalització, estabilitat, temps d'entrenament i memòria.
- Estudiar la influència dels principals hiperparàmetres sobre el procés d'aprenentatge, així com la funció de recompensa.
- Avaluar el rendiment dels agents en diversos entorns de complexitat progressiva.
- Aplicar els coneixements adquirits a la resolució simplificada d'un problema inspirat en la conducció autònoma, on l'agent hagi d'aprendre a controlar un vehicle dins d'un entorn simulat.

Mitjançant aquests objectius es pretén obtenir una visió global de les tècniques actuals d'aprenentatge per reforç i de les dificultats associades a la seva aplicació en problemes reals.

1.4 Metodologia

Tot i que Python s'ha consolidat com el llenguatge de referència en l'àmbit de l'aprenentatge automàtic gràcies al seu extens ecosistema de llibreries i la seva facilitat d'ús, presenta certes limitacions de rendiment en simulacions intensives, especialment quan es requereix executar un gran nombre d'episodis d'entrenament. Per aquest motiu, els algorismes desenvolupats en aquest treball s'implementaran en Rust, un llenguatge que combina elevades prestacions d'execució amb garanties de seguretat de memòria i eines eficients per a la programació concurrent.

Per a la simulació física dels entorns s'utilitzarà la llibreria [Rapier](#), un motor físic optimitzat escrit en Rust que permet executar simulacions de manera eficient i desacoblada del temps real, podent executar varis segons de simulació en un sol segon real.

La visualització es farà mitjançant plotters (una llibreria de generació de imatges) i [ffmpeg](#) per a la visualització a temps real.

1.4.1 Estructura de la memòria

Per assolir els objectius plantejats s'ha adoptat una metodologia iterativa basada en la implementació progressiva de diferents algorismes i entorns d'aprenentatge.

En primer lloc, es realitzarà una breu introducció als conceptes bàsics del RL, amb el qual es justificaran els components del framework propi implementat, que proporciona les abstraccions necessàries per desacoblar la definició dels entorns dels algorismes d'aprenentatge. Aquesta arquitectura permetrà reutilitzar components, facilitar l'experimentació i comparar diferents mètodes sota condicions equivalents.

Posteriorment, s'implementaran múltiples algorismes representatius del RL. Començarem amb el Q-Learning, utilitzat com a aproximació introductòria en entorns senzills i discrets. A continuació s'incorporarà DQN per superar les limitacions derivades de les taules de valors i permetre la generalització mitjançant xarxes neuronals. Finalment, s'implementaran PPO i mètodes evolutius, adequats per abordar problemes amb espais d'accions i estats continus i dinàmiques més complexes.

Cada algorisme s'avalua en una sèrie d'entorns de dificultat creixent, analitzant mètriques com la recompensa acumulada, la velocitat de convergèn-

cia i la robustesa davant diferents configuracions d'hiperparàmetres. Aquest procés permet identificar els punts forts i les limitacions de cadascun dels enfocaments estudiats.

Com a culminació del treball, els coneixements adquirits s'aplicaran a la construcció d'un entorn simplificat de conducció autònoma, on els agents hauran d'aprendre a controlar un vehicle a partir de la informació proporcionada pels sensors simulats. Aquest cas d'estudi serveix com a demostració pràctica de les capacitats del framework desenvolupat i dels algorismes implementats.

2 Fonaments Teòrics i Elements de RL

L'aprenentatge per reforç (Reinforcement Learning, RL) és un paradigma d'aprenentatge automàtic en què un agent aprèn a prendre decisions mitjançant la interacció amb un entorn. A cada instant, l'agent observa l'estat actual de l'entorn, selecciona una acció i rep una recompensa (positiva o negativa) que indica la qualitat de la decisió presa.

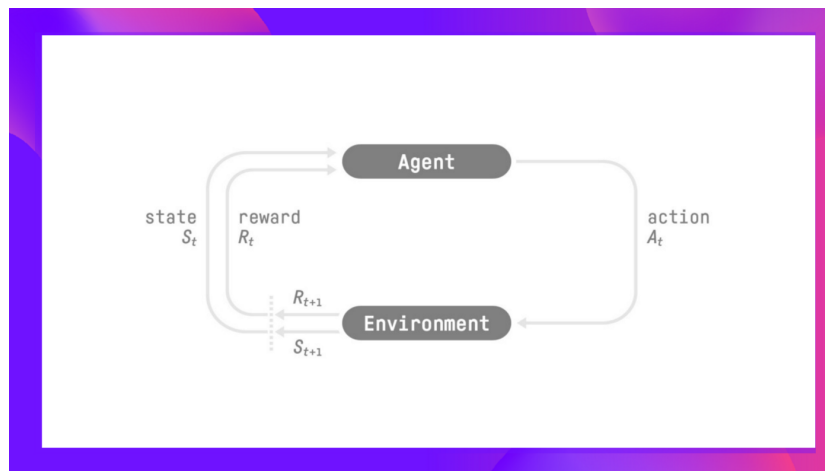


Figure 2.1: Bucle d'entrenament d'un algoritme de RL

Per tal d'aprendre, l'agent interactua amb l'entorn a base de prova i error, i utilitza el valor de la recompensa per aprendre. El seu objectiu és aprendre una estratègia que maximitzi la recompensa acumulada al llarg del temps. El procés de decisió de l'agent és la política π (i per tant, l'objectiu és trobar la política que maximitzi la recompensa acumulativa de l'episodi). Hi ha dos

tipus d'algorismes de RL:

- *Policy-based*: Aquests algorismes entrenen la política directament per aprendre quina acció prendre en cada casos
- *Value-based*: Aquests entrenen una funció que retorna el valor d'un estat en concret, i l'utilitzen per prendre l'acció que porta a l'estat de major valor.

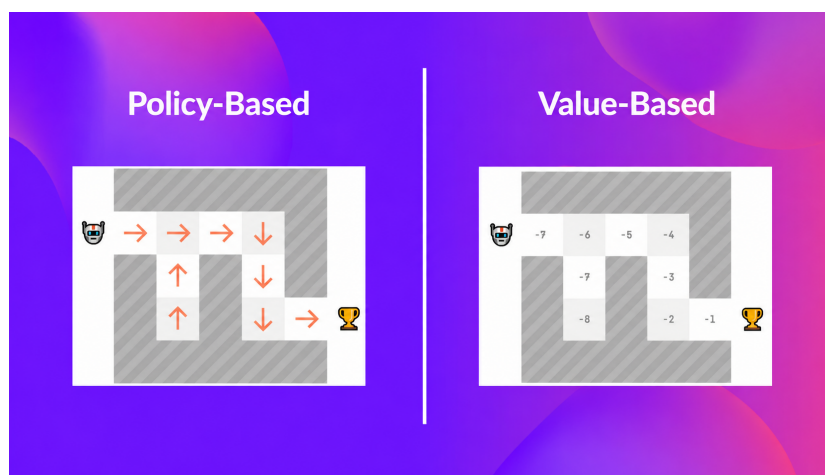


Figure 2.2: Diferència entre els principals algorismes de RL

2.1 Conceptes generals del RL

Agent L'agent és l'entitat encarregada de prendre decisions dins l'entorn.

A partir de la informació disponible sobre l'estat actual, selecciona una acció amb l'objectiu d'obtenir la màxima recompensa possible (o d'explorar l'entorn). En aquest treball, els agents corresponen als diferents algorismes implementats.

Entorn L'entorn representa el sistema amb el qual interactua l'agent. Aquest defineix les regles del problema, actualitza el seu estat en funció de les accions executades i proporciona les recompenses corresponents. Alguns exemples d'entorns són una graella de navegació, un laberint o la simulació d'una ciutat.

Estat L'estat és la informació que descriu la situació actual de l'entorn en un instant determinat. Aquesta informació és utilitzada per l'agent per decidir quina acció executar. La representació dels estats depèn del problema; per exemple, en una graella podria correspondre a les coordenades de l'agent, mentre que en un vehicle podria incloure la seva posició, velocitat i informació dels sensors. Decidir bé la informació que conté l'estat és clau per tal que l'agent pugui resoldre el problema. S'ha de tenir en compte que l'estat és l'únic que l'agent sap de l'entorn.

Acció Cada una de les decisions que l'agent pot executar sobre l'entorn. Segons el problema, l'espai d'accions pot ser discret o continu. Per exemple, en una graella les accions podrien ser moure's amunt, avall, esquerra o dreta, mentre que en la conducció autònoma poden correspondre a valors continus d'acceleració i direcció.

Recompensa (Reward) La recompensa és un valor numèric proporcionat per l'entorn després de cada acció. Aquest valor serveix com a mecanisme de retroalimentació i indica si l'acció realitzada ha estat beneficiosa o perjudicial per assolir l'objectiu. El disseny adequat de la funció de recompensa és un dels aspectes més importants en l'aprenentatge per reforç.

Política La política defineix el comportament de l'agent i determina quina acció s'ha de seleccionar en cada estat. Formalment, una política és una funció que associa estats amb accions o amb probabilitats sobre les accions disponibles. L'objectiu de l'entrenament és obtenir una política que maximitzi la recompensa esperada.

Episodi Un episodi és una seqüència completa d'interaccions entre l'agent i l'entorn. Comença en un estat inicial i finalitza quan es compleix una condició de terminació, com ara assolir l'objectiu o superar un nombre màxim de passos. L'aprenentatge es basa habitualment en l'experiència acumulada al llarg de múltiples episodis.

Exploració vs Explotació Un dels principals reptes de l'aprenentatge per reforç és trobar un equilibri entre exploració i explotació. L'exploració consisteix a provar accions noves per obtenir informació sobre l'entorn, mentre que l'explotació consisteix a seleccionar les accions que l'agent

considera més beneficioses segons el coneixement adquirit fins al moment. Un excés d'exploració pot alentir l'aprenentatge, mentre que un excés d'explotació pot conduir a solucions subòptimes.

2.2 Mètodes basats en la funció de valor

El primer algorisme que implementarem serà el Q-Learning, un algorisme basat en trobar la funció de valor. En aquesta secció explorarem els fonaments teòrics que assegurin el seu funcionament.

2.2.1 Relació entre els mètodes basats en valor vs els basats en política

En els mètodes basats en valor, el que s'aprèn és una funció que mapeja cada estat al valor esperat d'estar en aquell estat, on el valor esperat d'un estat és la recompensa esperada descomptada que l'agent pot aconseguir si comença a un estat, i actua d'acord amb la política trobada:

$$\underbrace{v_{\pi}(s)}_{\text{Valor de l'estat}} = \mathbb{E}_{\pi} \underbrace{\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]}_{\text{Retorn descomptat esperats}} \quad (2.1)$$

on

- t correspon a l'instant temporal actual.
- k indica quants passos en el futur ens trobem respecte a l'instant actual t . Així, $k = 0$ correspon a la recompensa immediata (R_{t+1}), $k = 1$ a la següent (R_{t+2}), etc.
- $\gamma \in [0, 1]$ és el *factor de descompte*, que determina la importància de les recompenses futures. Com més proper a 1 sigui γ , més es valoren les recompenses llunyanes; com més proper a 0, més pes tenen les recompenses immediates.
- R_{t+k+1} és la recompensa obtinguda després de realitzar la transició corresponent al pas $t + k$.

- $\mathbb{E}_\pi[\cdot]$ representa l'esperança matemàtica assumint que l'agent segueix la política π .
- $S_t = s$ indica que l'agent es troba a l'estat s en l'instant t .

Però recordem que aquests mètodes no troben directament una política, sinó que només tenen la funció del valor de cada estat. Ens queda definir el comportament de la política $\pi(s)$. Com volem que aquesta prengui les accions que sempre ens portin a la major recompensa, crearem la *política gredy*:

$$\pi(s) = \arg \max_a Q_\pi(s, a)$$

- $\pi(s)$ retorna l'acció seleccionada per la política en l'estat s .
- $Q_\pi(s, a)$ és el valor esperat de realitzar l'acció a en l'estat s i continuar seguint la política π .
- $\arg \max_a$ retorna l'acció que maximitza el valor de $Q_\pi(s, a)$.

2.2.2 Equació de Bellman

Ara ja sabem trobar la política a partir de la funció de valor de cada estat. Encara ens queda saber com calcular el valor de cada estat. Això és el que resol l'equació de Bellman. La seva idea principal és que el valor d'un estat es pot descompondre en dos components: la recompensa immediata obtinguda en el següent pas i el valor esperat dels estats futurs. Aquesta propietat, coneguda com a *principi d'optimalitat de Bellman*, permet expressar problemes seqüencials complexos mitjançant una relació recursiva.

En altres paraules, en lloc d'avaluar directament totes les possibles seqüències futures d'accions, és possible calcular el valor d'un estat a partir de la informació disponible en els seus estats successors. Aquesta formulació és la base de molts algorismes de programació dinàmica i de diversos mètodes de Reinforcement Learning, com ara Q-Learning o Deep Q-Networks.

Per a una política π , el valor d'un estat es defineix com l'esperança del retorn acumulat descomptat que s'obtindrà seguint aquesta política. L'equació de Bellman per a la funció de valor d'estat és (es pot obtenir desenvolupant

l'equació 2.1):

$$\underbrace{v_\pi(s)}_{\text{Valor de l'estat}} = \mathbb{E}_\pi \left[\underbrace{R_t}_{\substack{\text{Retorn} \\ \text{immediat} \\ \text{esperat}}} + \underbrace{\sum_{k=1}^{\infty} \gamma^k R_{t+k+1}}_{\substack{\text{Valor descomptat} \\ \text{del següent estat}}} \mid S_t = s \right] = \mathbb{E}_\pi[R_{t+1} + \gamma \cdot v_\pi(s_{t+1}) \mid S_t = s] \quad (2.2)$$

Aquesta equació mostra que el valor d'un estat és igual a la recompensa immediata esperada més el valor futur esperat, ponderat pel factor de descompte. Gràcies a aquesta relació recursiva, és possible estimar iterativament els valors dels estats fins a convergir cap a una solució estable.

2.3 Framework de RL

A partir dels conceptes introduïts a la secció anterior, és possible identificar les funcionalitats bàsiques que han de proporcionar tant els entorns com els algorismes de RL. Aquest anàlisi permet definir una interfície comuna que facilita la reutilització de components i la comparació entre diferents mètodes.

2.3.1 Entorn

- **get_state():** És clar que l'entorn ha de ser capaç de donar l'estat actual (el que l'agent pot veure)
- **get_possible_actions():** Retorna el conjunt d'accions disponibles en l'estat actual. Aquesta funció és especialment útil en entorns amb espais d'accions discrets.
- **step(Acció):** Aplica l'acció indicada, actualitza l'estat de l'entorn (modificant la posició de l'agent, per exemple) i retorna la recompensa obtinguda juntament amb la informació necessària per determinar si l'episodi ha finalitzat.
- **reset():** Reiniciar l'entorn a l'estat inicial per començar un nou episodi

2.3.2 Algorisme

- **train_epoch(Entorn)**: Executa un episodi complet d'entrenament i actualitza els paràmetres interns de l'algorisme a partir de l'experiència obtinguda.
- **bset_action(Estat, Accions)**: Obté la millor acció donat l'estat actual i les possibles accions (i la política actualment apresada)

Com és de preveure, dins de la funció **train_epoch(Entorn)** trobem el bucle que demana a l'entorn l'estat actual, escull una opció (per explorar o explotar) i avança l'entorn en un step aplicant l'acció escollida.

El bucle d'entrenament és senzillament la repetició de l'entrenament d'un episodi.

Listing 2.1: Bucle principal d'entrenament

```
for k in 0..n_epochs {  
    algo.train_epoch(&mut environment);  
}
```

Aquestes interfícies es materialitzen en el framework mitjançant els traits `EnvironmentTrait` i `AlgorithmTrait`, que permeten desacoblar completament la implementació dels entorns dels algorismes d'aprenentatge (les definicions esmentades es troben al fitxer `src/rl/mod.rs`).

3 Primer contacte: Entorn Discret

Recordem que l'objectiu d'aquest treball és tant entendre les bases del reinforcement learning com acabar desenvolupant un agent capaç de conduir autònomament. Per tant, té sentit començar amb l'entorn més senzill possible: una graella, amb un inici i una cel·la objectiu. Això permet tant començar a treballar amb els algorismes més bàsics (per aprofundir en els seus mecanismes d'aprenentatge) com tenir un entorn base on provar si els algorismes més complexos funcionen adequadament (un entorn molt senzill i interpretable).

Aquest entorn senzill permetrà estudiar molts dels conceptes fonamentals de l'aprenentatge per reforç: l'exploració i explotació, el disseny de recompenses, la representació dels estats i la convergència dels algorismes d'aprenentatge.

A més, aquest problema comparteix certes característiques amb la conducció autònoma. En ambdós casos l'agent ha de prendre una seqüència de decisions amb l'objectiu d'arribar a una destinació determinada minimitzant el cost associat al trajecte. Per aquest motiu, la graella constitueix un primer pas adequat abans d'introduir entorns més realistes.

3.1 Implementació de la Graella

A l'hora de parlar de la implementació de tot entorn, hem de fer referència a les 4 funcions descrites al Framework:

- Quin és l'estat que l'entorn presentarà a l'agent
- Quines són les possibles accions en l'entorn

- Com és l'estructura de dades de l'entorn? Quina informació guarda? Què es necessita per reiniciar-lo?
- Què implica realitzar un pas en l'entorn, i com es calcula la funció de reward

3.1.1 Estat

Recordem que els únics paràmetres d'aquest entorn són les coordenades de la cel·la on es troba l'agent i les coordenades d'on es troba l'objectiu. Per tant, podem utilitzar com a estat el vector 2d direcció a l'objectiu (aquesta és tota la informació que necessita l'agent per decidir quina acció realitzar).

3.1.2 Acció

Donat que estem en una graella, l'agent haurà de triar entre 4 accions:

- Anar a la cel·la superior $(+(0, 1))$
- Anar a la cel·la inferior $(-(0, 1))$
- Anar a la cel·la de l'esquerra $(-(1, 0))$
- Anar a la cel·la de la dreta $(+(1, 0))$

3.1.3 Estructura de dades

Aquest és un entorn senzill, i necessita pocs paràmetres:

- Posició actual de l'agent $([i32, i32])$ i de la cel·la objectiu $([i32, i32])$
- Posició inicial de l'agent $([i32, i32])$: Per reiniciar l'entorn
- Amplada $(i32)$ i alçada $(i32)$ de la graella: Per assegurar-nos que l'agent no surt de la graella
- Murs $Set([i32, i32], Acció)$: Murs que limiten el moviment. Utilitzat per convertir la graella en un laberint més tard
- Tipus de reward: Pla vs Gradient (veure [3.1.5](#))

3.1.4 Pas

Fer un pas és senzill: simplement s'ha de sumar el vector direcció associat a cada acció a la posició actual de l'agent.

3.1.5 Reward

Recordem que aquesta ha de dir a l'agent si l'acció presa és bona o dolenta. Per tant, és clar que arribar a la cel·la objectiu tindrà una recompensa positiva elevada. Però hem d'afegir també una penalització per cada pas que l'agent faci (si no, qualsevol camí que portés a l'objectiu tindria la mateixa recompensa, independentment de la llargada del camí, i ens interessa que trobi el camí més curt). Aquí veiem que ho podem fer de dues maneres:

- Un reward pla, on a cada pas restem la mateixa quantitat
- Una recompensa més informada, on a cada pas el valor és proporcional a la distància de Manhattan a l'objectiu

Més endavant veurem com afecta aquesta funció de recompensa durant l'entrenament.

Com a comentari, els valors concrets es decidiran perquè es compleixi el següent criteri:

- El reward obtingut si se segueix el camí més curt és 0
- El reward obtnigut si es recorren totes les cel·les (sense tenir en compte la recompensa obtinguda al arribar a la cel·la objectiu) és de -100

Aquests criteris són per poder comparar les dues funcions de reward a la mateixa gràfica, sense que l'escala de cada funció concreta afecti a la comprensió visual.

3.2 Primer algorisme: Q-Learning

Q-Learning és un dels algorismes més coneguts de l'aprenentatge per reforç. Es tracta d'un mètode basat en trobar una funció de valor (veure secció 2.2).

La idea principal consisteix a estimar una funció d'acció-valor $Q(s, a)$, que representa la recompensa acumulada esperada d'executar una acció a en un estat s i continuar posteriorment seguint la millor política coneguda.

Durant l'entrenament, els valors de la taula Q s'actualitzen iterativament segons l'equació de Bellman (2.2), aplicant els principis de *Temporal Difference Learning* (és a dir, aprenent després de prendre cada acció):

$$\underbrace{Q(s, a)}_{\substack{\text{Nou valor} \\ \text{de l'estat } s \\ \text{i acció } a}} \leftarrow \underbrace{Q(s, a)}_{\substack{\text{Estimació} \\ \text{prèvia}}} + \underbrace{\alpha}_{\substack{\text{Lerning} \\ \text{rate}}} \left[\underbrace{R_{t+1}}_{\substack{\text{Reward}}} + \underbrace{\gamma V(S_{t+1})}_{\substack{\text{Valor descomptat} \\ \text{del següent} \\ \text{estat}}} - Q(s, a) \right] \quad (3.1)$$

On $V(S_{t+1}) = \max_a Q(s, a)$

Aquesta expressió permet actualitzar progressivament el coneixement de l'agent a mesura que interactua amb l'entorn, construint el que s'anomena una *Q-table*, que emmagatzema una estimació del valor de cada parell estat-acció. Per a cada estat possible existeix una entrada per cadascuna de les accions disponibles. Durant l'entrenament, els valors de la taula es modifiquen segons les recompenses observades fins que convergeixen cap a una aproximació de la política òptima.

L'avantatge principal d'aquest enfocament és la seva simplicitat. Tanmateix, el nombre d'entrades creix proporcionalment al nombre d'estats multiplicat al nombre d'accions, fet que dificulta la seva aplicació en problemes grans o continus.

3.2.1 Pseudocodi

Algorithm 1 Tabular Q-Learning: entrenament d'un episodi

Require: Entorn E , taula Q , paràmetres α , γ , ϵ , $\epsilon_{\min}$, màxim de passos N

```
1:  $steps \leftarrow 0$ 
2:  $s \leftarrow E.get\_state()$ 
3:  $A \leftarrow get\_all\_actions()$ 
4: while true do
5:    $a \leftarrow EPSILONGREEDYACTION(s, A, Q, \epsilon)$ 
6:    $(r, terminated) \leftarrow E.step(a)$ 
7:   if  $terminated$  or  $steps > N$  then
8:      $Q(s, a) \leftarrow Q(s, a) + \alpha(r - Q(s, a))$ 
9:      $\epsilon \leftarrow \max(\epsilon \cdot d, \epsilon_{\min})$ 
10:     $E.reset()$ 
11:    break
12:  else
13:     $s' \leftarrow E.get\_state()$ 
14:     $q_{\max} \leftarrow \max_{a' \in A} Q(s', a')$ 
15:     $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma q_{\max} - Q(s, a))$ 
16:     $steps \leftarrow steps + 1$ 
17:     $s \leftarrow s'$ 
18:  end if
19: end while
```

Algorithm 2 Selecció d'acció greedy amb ϵ -greedy

Require: Estat s , conjunt d'accions A , taula Q , probabilitat d'exploració ϵ

```
1: if rand() <  $\epsilon$  then
2:   Retornar una acció aleatòria de  $A$ 
3: else
4:    $best \leftarrow \emptyset$ 
5:    $v_{\max} \leftarrow -\infty$ 
6:   for all  $a \in A$  do
7:      $v \leftarrow Q(s, a)$ 
8:     if  $v > v_{\max}$  then
9:        $v_{\max} \leftarrow v$ 
10:       $best \leftarrow \{a\}$ 
11:    else if  $|v - v_{\max}| < \delta$  then
12:      Afegir  $a$  a  $best$ 
13:    end if
14:  end for
15:  Retornar una acció aleatòria de  $best$ 
16: end if
```

3.2.2 Detalls de la implementació

Si bé es pot implementar la Q -table com una matriu amb columnes el nombre possibles d'estat, i files el nombre possibles d'accions, he preferit implementar-la com un diccionari on la clau és precisament la tupla (S, A) i el valor, el valor esperat.

3.2.3 Estudi de l'algorisme a l'entorn discret

Aquest primer entorn i algorisme de RL és relativament simple, i molt útil per estudiar-lo i poder treure conclusions per tal d'aplicar-les en entorns més complexos.

Estudi dels hiperparàmetres

Hem vist al pseudocodi (1) que hi havia 3 hiperparàmetres principals: el learning rate (α), el factor de descompte de les recompenses (γ) i un paràmetre que regula l'exploració vs. l'explotació (ϵ). Ara veurem com afecta cada

un d'aquests hiperparàmetres en l'aprenentatge d'una graella 10×10 , on l'agent es troba inicialment a la posició $(0,0)$ i l'objectiu a la $(9,9)$.

Learning Rate vs Reward Discount Factor

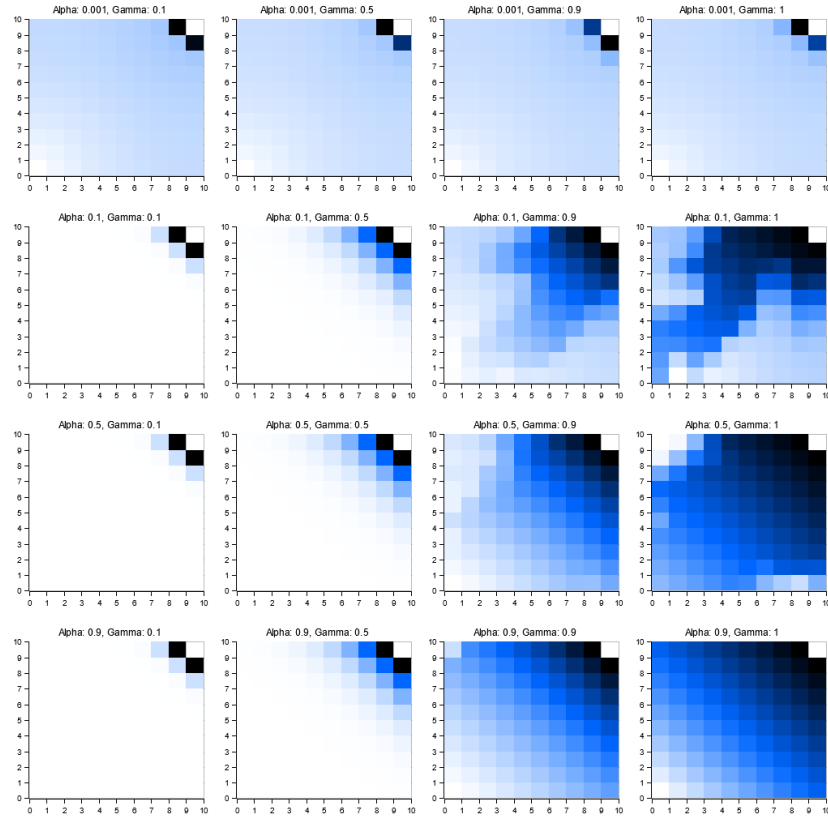


Figure 3.1: $V(S)$ per cada cel·la al finalitzar l'entrenament

A partir de la imatge prèvia podem veure clarament els efectes del learning rate i del factor de descompte de la recompensa (que també podem preveure a partir de la teoria):

- Primera i segona columna $\rightarrow \gamma$ baix: Les recompenses es propaguen molt poc. És a dir, l'agent és miop. Si hi haguessin altres recompenses menors a la graella, l'agent tendiria a anar a les més properes (encara que n'hi hagués d'altres que donen més recompenses)

- Tercera i quarta columna $\rightarrow \gamma$ alt: Ara es veu clarament un gradient de la funció de valor que porta a l'objectiu
- Primera fila $\rightarrow \alpha$ molt baix: Pràcticament tots els estats tenen el mateix valor. És a dir, l'algorisme no ha après.
- Segona fila $\rightarrow \alpha$ baix: Aquí ja veiem un gradient de valors cap a l'objectiu. Per tant aquest seria el mínim learning rate acceptable
- Tercera i quarta fila $\rightarrow \alpha$ alt: També semblent bons lerning rates (no apareixen illes, i el valor d'una cel·la es propaga a les veïnes). Aquest deu ser un entorn prou simple perquè un learning rate agressiu no produeixi oscil·lacions

Exploració vs explotació tradeoff

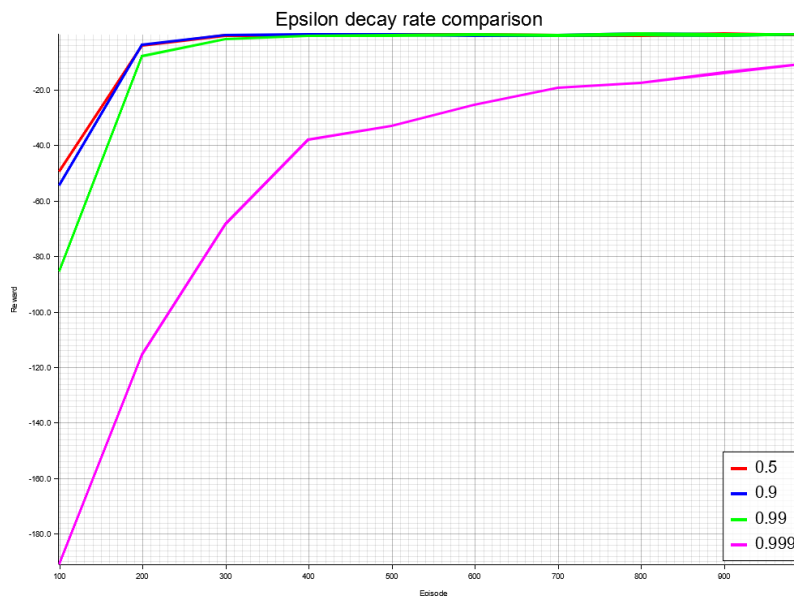


Figure 3.2: Reward de cada episodi per diferent ϵ

Aquest resultat era esperable:

- ϵ petits fan que l'algorisme deixi d'explorar ràpidament (i en aquest entorn, com és molt senzill (hi ha 18 passos fins a l'objectiu), això vol dir que l'algorisme troba la millor trajectòria ràpidament)
- ϵ molt elevat: L'algorisme encara està explorant. Si calculem $0.999^{1000} \approx 0.37$, és a dir, encara escull 1 terç de les accions de forma aleatòria.

Estudi de la funció de reward

Hem comentat a la secció (3.1.5) que hi havia 2 possibles funcions de reward: una on el reward a cada casella és el mateix, i una on augmenta a mesura que ens apropem a l'objectiu. Ara veurem com afecta a la velocitat d'entrenament:

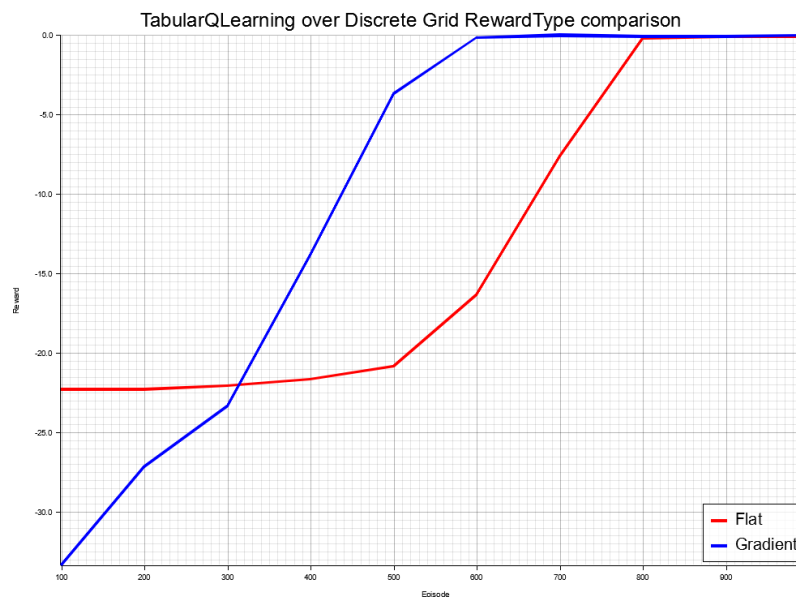


Figure 3.3: Reward de cada episodi per diferent tipus de recompensa

Notem que la intuïció no fallava: un reward més informat porta a l'agent a aprendre la millor política amb menys iteracions. Aquest és un entorn senzill, però és d'esperar que en un entorn on sigui més difícil arribar a l'objectiu de forma aleatòria, un reward en forma de gradient a l'objectiu realment afavoreixi l'aprenentatge.

Prova en un entorn lleugerament més complex

És interessant veure que el Q-learning pot resoldre problemes que s'acostumen a plantejar utilitzant altres algorismes, com per exemple la resolució d'un laberint:

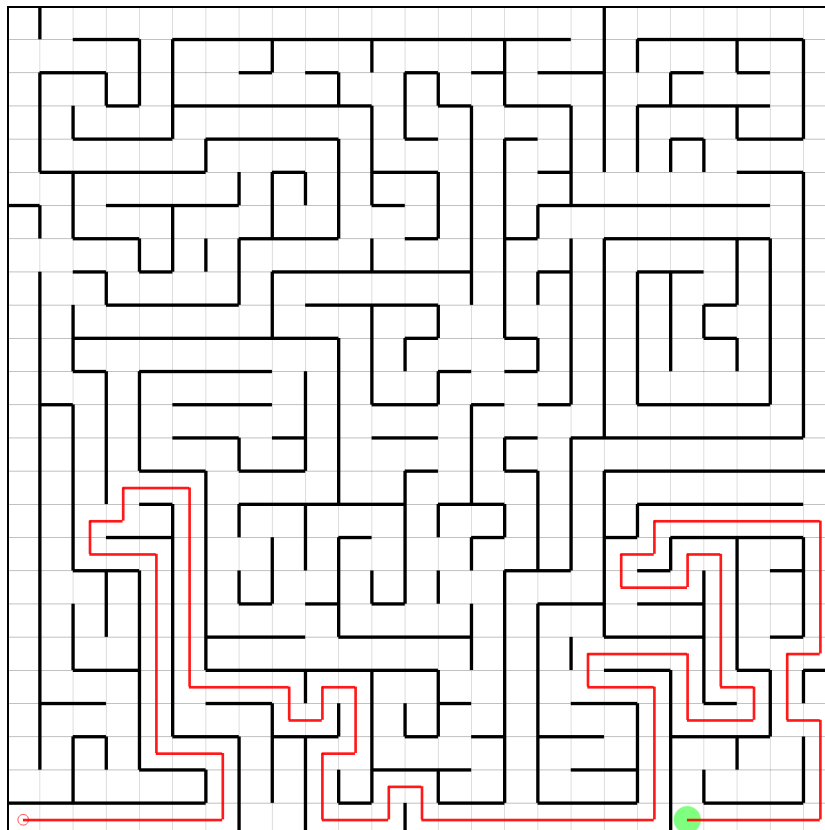


Figure 3.4: Prova que el Q learning és capaç de resoldre laberints

Tot i poder-los resoldre, s'utilitzaen altres algorismes per resoldre aquestes tasques perquè són més eficients. A més, requereix tocar els hiperparàmetres del Q-learning per tal que arribi a poder resoldre'ls (a diferència del BFS, per exemple, que no té hiperparàmetres a modificar). Però és interessant veure que els algorismes de RL poden resoldre diferents tipus de tasques, sempre que tinguin l'entorn ben definit.

Com a comentari, els hiperparàmetres els he modificat de forma que l' ϵ arribi pràcticament a 0 al final de l'entrenament, de manera que l'algorisme

explori la major part del temps (a més, el tipus de reward en aquest problema és flat, donat que per utilitzar el gradient a l'objectiu necessitariem saber el camí a aquesy, que és precisament el que estem buscant)

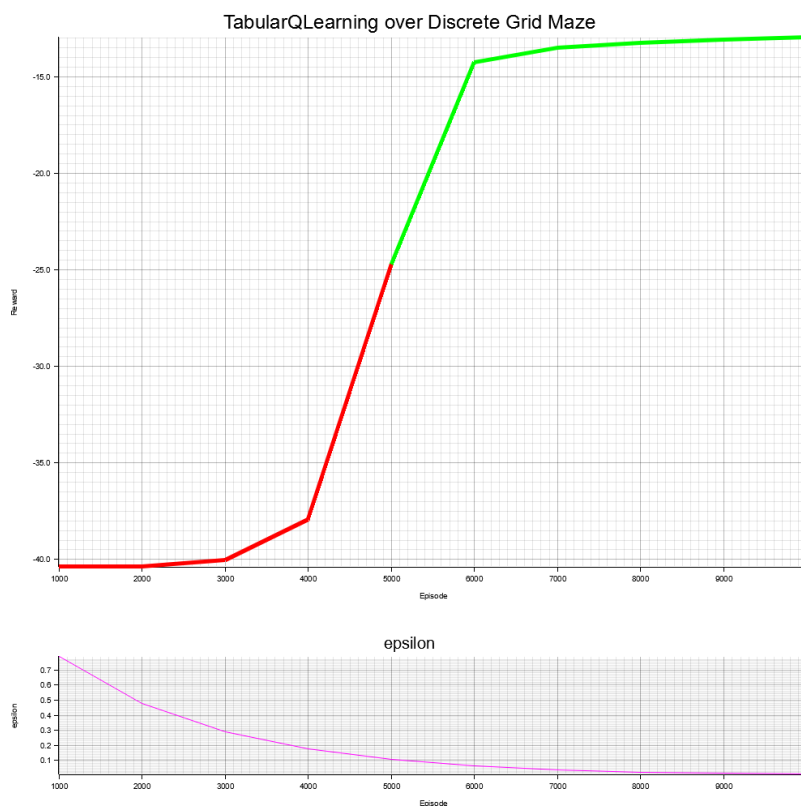


Figure 3.5: Reward vs episodi a l'entrenament al laberint anterior (on la línia vermella informa que no s'ha arribat a l'objectiu a l'episodi i , i la verda que sí), juntament amb el valor de l'èpsilon

Limitacions

La primera limitació que podem veure del tabular-q-learning és la seva incapacitat de generalització. Si una tupla (Estat, Acció) no es troba al seu diccionari, no sap el seu valor i conseqüentment no sap com actuar. Això ho podem visualitzar a través de la següent imatge. Per realitzar-la, hem fet el següent:

1. He entrenat l'algorisme en una graella 10×10 (utilitzant hiperparàmetres adequats segons els vistos a la subsecció 3.2.3)
2. He provat de seguir la millor política en un entorn 15×15

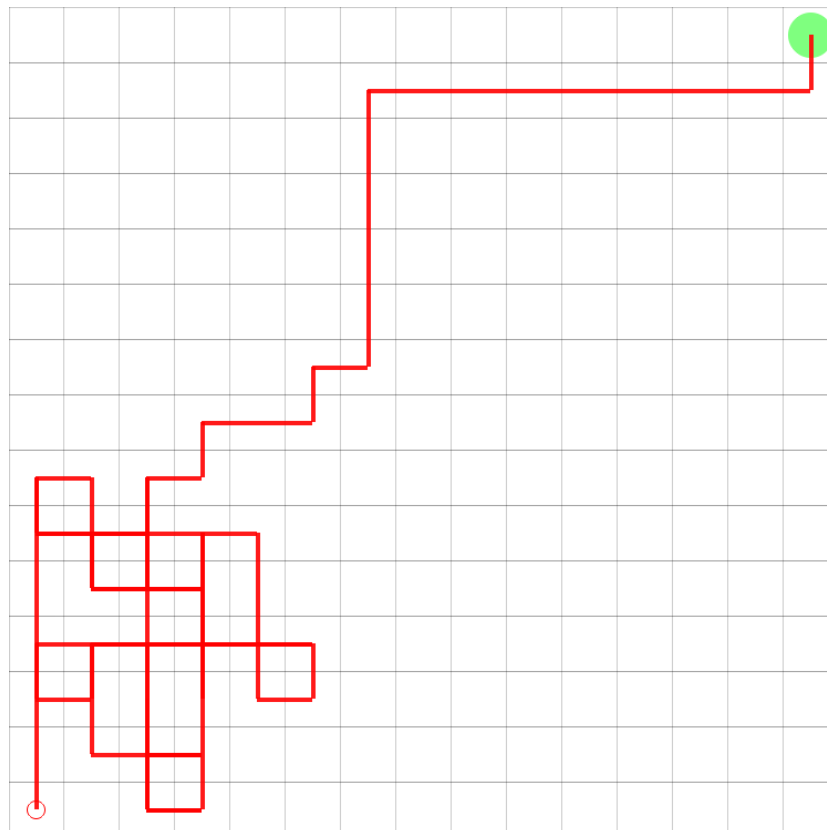


Figure 3.6: Trajectòria d'una Q-table testejada en una graella major que la de l'entrenament

Veiem que l'agent actua de forma aleatòria fins que es troba en una subgraella 10×10 de l'objectiu, on aleshores ja té apresada la millor política i sap arribar a l'objectiu

Una altra limitació que podem deduir del tabular Q learning és que l'ús de memòria s'incrementa proporcionalment amb el nombre d'estats pel nombre d'accions. Per això només és possible aplicar aquest algorisme a entorns discrets amb accions discretes.

4 Entorn continu

Fins ara hem vist els fonaments teòrics, i un primer algorisme que ens ha servit per entendre els hiperperàmetres generals dels models de EL. Ara és moment de passar a un entorn més complex, estudiar una nova generació d'algorismes que permetin l'entrada d'un estat no finit i triar-ne un per aplicar-lo a la conducció autònoma.

4.1 Implementació de l'entorn continu

Aquest nou entorn serà un pla 2D, sobre el qual l'agent es podrà moure, amb l'objectiu d'arribar a una posició determinada. On podrem posar-hi obstacles que l'agent haurà d'esquivar.

Per poder simular l'entorn, he utilitzat el motor físic Rapièr. El motiu d'utilitzar un motor (quan l'entorn és prou simple) són els següents: per una banda, aquest motor ja té incorporats funcions per veure col·lisions (entre l'agent i els obstacles, i els rajos amb els obstacles). Per l'altre, era una manera de familiaritzar-me amb ell, ja que el seu ús serà més intensiu en la simulació d'un vehicle.

4.1.1 Estat

Com en la graella, necessitem que l'agent tingui una idea d'on està l'objectiu. En aquest cas, en comptes de donar-li a l'agent el vector direcció a l'objectiu m'he decantat per donar-li la distància (un nombre) i l'angle. A més, com després hi afegirem obstacles, he afegit uns sensors (nombres $\in [0, 1]$) que veuen si hi ha objectes a les proximitats.

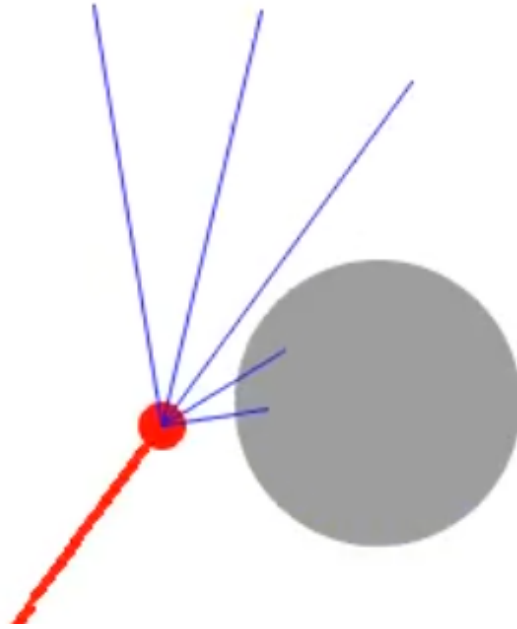


Figure 4.1: Imatge on es visualitzen els sensors que ofereixen a l'algorisme la informació dels obstaclesm és propers (els ulls)

4.1.2 Acció

En aquest cas, he obtat per donar-li a l'agent 3 possibles accions:

- Fer un pas endavant ($1/60$ unitats)
- Girar cap a l'esquerre ($1/60$ parts de volta)
- Girar cap a la dreta ($1/60$ parts de volta)

4.1.3 Estructura de dades

Aquest entorn ha de guardar en memòria:

- Informació estàtica de l'entorn, com la posició dels obstacles, la de l'objectiu i la inicial de l'agent.

- La simulació física en sí. En ella s'actualitza la posició de l'agent modificant o bé la seva velocitat lineal o la seva velocitat angular

4.1.4 Pas

Fer un pas és senzill: és veure quina acció ha seleccionat l'algorisme i canviar la velocitat de l'agent en conseqüència:

```
let rot: Quat = *agent.rotation();
let forward: Vec3 = rot * Vector3::Z;
match action {
    Environment2dActions::Forward => agent.set_linvel(
        forward),
    Environment2dActions::TurnLeft => agent.set_angvel(
        Vector::new(0.0, -2.0 * PI, 0.0)),
    Environment2dActions::TurnRight => agent.set_angvel(
        Vector::new(0.0, 2.0 * PI, 0.0)),
}
```

4.1.5 Reward

Aquí tenim els mateixos dos tipus de reward presentats a la secció 3.1.5: Pla i Gradient. Per defecte, utilitzaré el reward de tipus Gradient (hi ha un sol punt objectiu, i trobo poc probable que de forma totalment aleatòria hi arribin). Quan tingui un algorisme que funcioni bé en aquest nou entorn, provaré el Reward de tipus pla, com a curiositat, per veure si l'agent és capaç igualment de solucionar l'entorn.

Els valors del reward en Gradient han estat creats perquè sempre estiguin entre 0 i 1.

4.2 Algorisme: Deep Q Learning

Recordem de la secció 3.2, que el tabular Q-learning (TQL) funcionava construint una graella amb la tupla (Estat, Acció) com índexs. El problema és precisament que es necessita construir aquesta taula, on ha d'aparèixer cada Estat i Acció. Això esdevé un problema si la dimensió d'entrada és prou gran (com al nostre cas, on podem dir que hi ha infinits punts entre 2 punts del pla, si ignorem per un moment la precisió de punt flotant de l'ordinador).

Aquí és on apareix la idea del Deep Q Learning, l'evolució natural del Tabular Q Learning. Aquest nou algorisme consisteix essencialment en una xarxa neuronal, on li entra un estat (la dimensió d'entrada ha de ser igual a la de l'estat) i té com a última capa tants nodes com accions pot escollir.

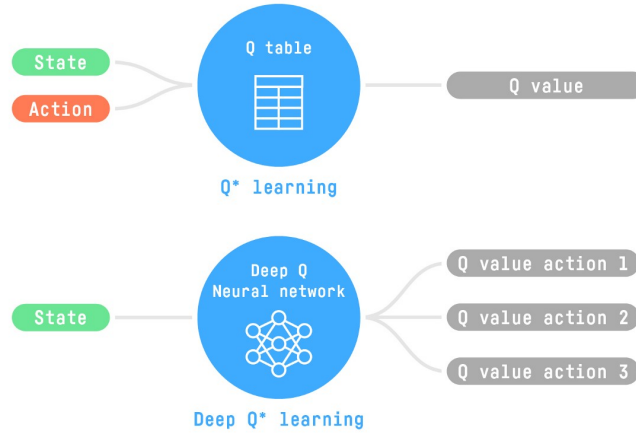


Figure 4.2: Diferència entre el Tabular-Q-Learning i el Deep-Q-Learning

Ara queda resoldre la qüestió de com actualitzar els pesos. En el TQL, aplicavem directament l'equació de Bellman (en l'aprenentatge per diferències temporals) per actualitzar els valors de la taula (3.1). Donat que a l'optimitzador és on s'aplica el learning rate, podem pensar que serà una bona idea utilitzar com a loss la part que multiplicava el learning rate a l'equació del TQL, és a dir:

$$\underbrace{R_{t+1}}_{\text{Reward}} + \gamma \underbrace{V(S_{t+1})}_{\text{Valor de la xarxa al següent estat}} - \underbrace{Q(s, a)}_{\text{Valor de la xarxa al actual estat}} \quad (4.1)$$

4.2.1 Pseudocodi i execució

Algorithm 3 Deep Q Learning: Entrenament d'un episodi

Require: Entorn E , xarxa Q_θ , Optimitzador, paràmetres α , γ , ϵ , $\epsilon_{\min}$,
màxim de passos N

```
1:  $s \leftarrow E.get\_state()$ 
2:  $A \leftarrow get\_all\_actions()$ 
3: while true do
4:    $q\_values \leftarrow Q_\theta(state)$ 
5:   if  $rand() < \epsilon$  then
6:      $action \leftarrow$  acció aleatòria
7:   else
8:      $action \leftarrow \arg \max_a Q_\theta(state, a)$ 
9:   end if
10:   $(reward, terminal) \leftarrow env.step(state, action)$ 
11:  if  $terminated$  or  $steps > N$  then
12:     $target \leftarrow reward$ 
13:     $td\_error \leftarrow target - Q_\theta(state, action)$ 
14:     $loss \leftarrow td\_error^2$ 
15:    Actualitza  $\theta$  amb backpropagation sobre  $loss$ 
16:    break
17:  else
18:     $next\_state \leftarrow E.get\_state()$ 
19:     $target \leftarrow reward + \gamma \cdot \max_a Q_\theta(next\_state, a)$ 
20:     $td\_error \leftarrow target - Q_\theta(state, action)$ 
21:     $loss \leftarrow td\_error^2$ 
22:    Actualitza  $\theta$  amb backpropagation sobre  $loss$ 
23:     $steps \leftarrow steps + 1$ 
24:     $state \leftarrow next\_state$ 
25:  end if
26: end while
```

Si executem aquesta primera versió de DQL en una graella 10×10 , sense obstacles:

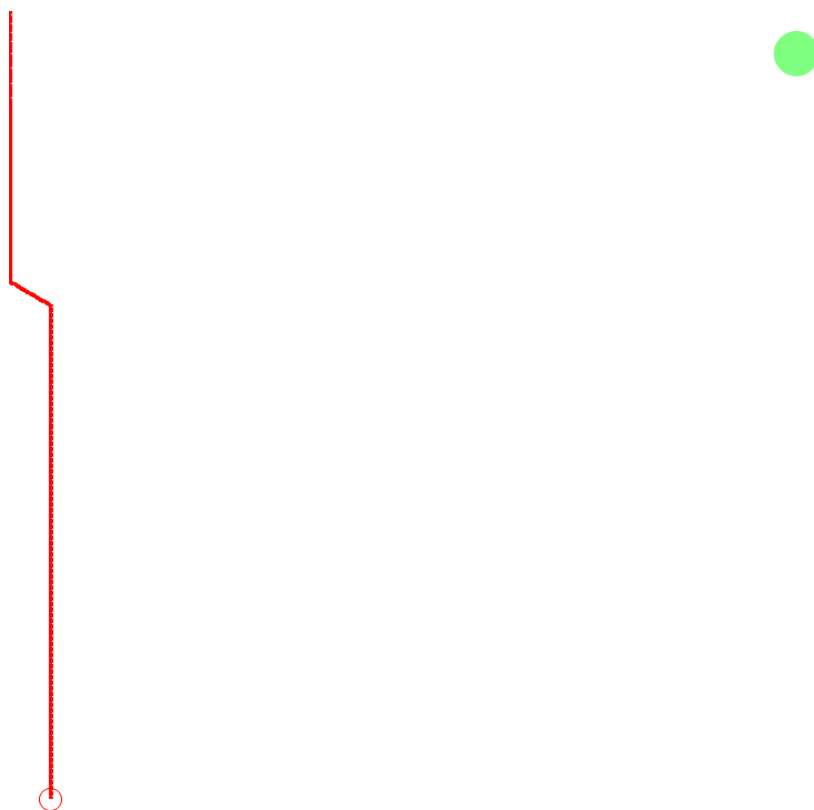


Figure 4.3: Execució del DQN "naive"

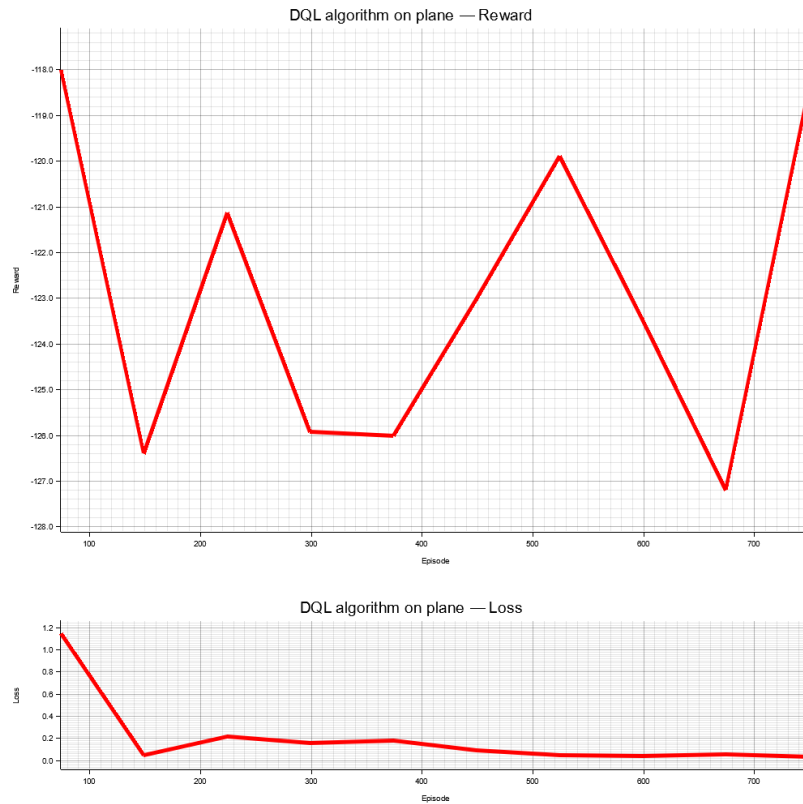


Figure 4.4: Mètriques del DQN "naive"

Veiem que falla estrepitosament: L'agent no és capaç de trobar l'objectiu.

Observant la gràfica de mètriques, podem extreure algunes conclusions (possibles motius pels quals això passa) i proposar solucions:

Millora: Replay buffer

Notem que la gràfica de la loss és una mica inestable (a l'episodi 150 arriba a un mínim, llavors puja i oscil·la abans de tornar a baixar). Un possible motiu és que la xarxa no està aprenent correctament.

Les xarxes neuronals necessiten que les entrades que li arribin estiguin poc correlacionades entre elles. Això no passa en la configuració actual de l'algorisme, donat que estats consecutius estan directament correlacionats (el següent depèn completament de l'anterior).

La solució acceptada és la següent: se separa l'entrenament d'un episodi en dues parts:

1. **Rollout:** En aquesta primera part l'agent interactua amb l'entorn i emmagatzema les experiències (estats, accions, si ha completat l'entorn, ...) en una memòria
2. **Lear:** Seguidament, s'agafen de forma aleatòria i desordenada N experiències i s'entrena la xarxa neuronal utilitzant aquestes. D'aquesta manera eliminem aquesta correlació entre entrades consecutives.

Aquesta memòria (*Replay Buffer*) ofereix l'avantatge que pot emmagatzemar múltiples episodis, reutilitzant experiències anteriors.

Millora: Q-Target

També observem inestabilitat en la funció de reward. És a dir, els valor de la Q-network (la xarxa neuronal de l'agent) no para d'oscil·lar. Una possible explicació és que estem utilitzant la mateixa xarxa neuronal per calcular els valors esperats (és a dir, utilitzem els mateixos pesos per calcular el Q-valor i td_target (el valor esperat)). A cada passa, ambdós valors canvien. És a dir, a cada passa, ens apropem al valor esperat, però aquest al seu torn també es mou. Això pot crear oscil·lacions en l'entrenament.

La solució proposada és crear una segona xarxa neuronal, amb la mateixa estructura que la xarxa neuronal principal i inicialitzada amb els mateixos pesos. Aquesta serà la xarxa que s'utilitzarà com a valor objectiu. La gràcia és que aquesta només s'actualitzarà (copiant els pesos de la Q-network actual) cada n passes, fixant el valor objectiu i permetent que la Q-network s'apropi sense oscil·lar.

Execució amb les noves modificacions

Si ara tornem a executar l'algorisme amb les modificacions descrites sobre el mateix entorn anterior:

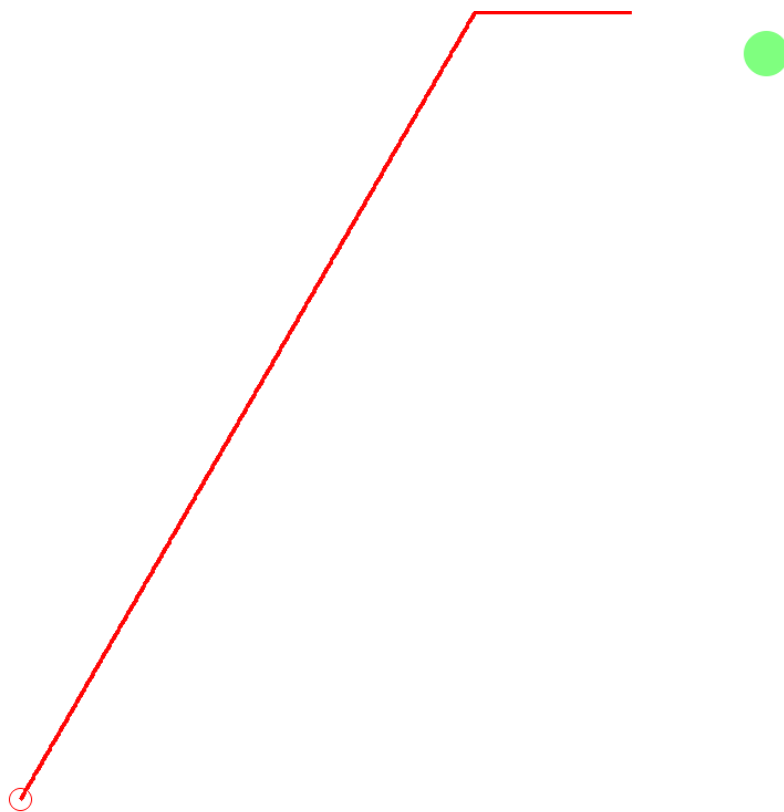


Figure 4.5: Execució del DQN amb millores

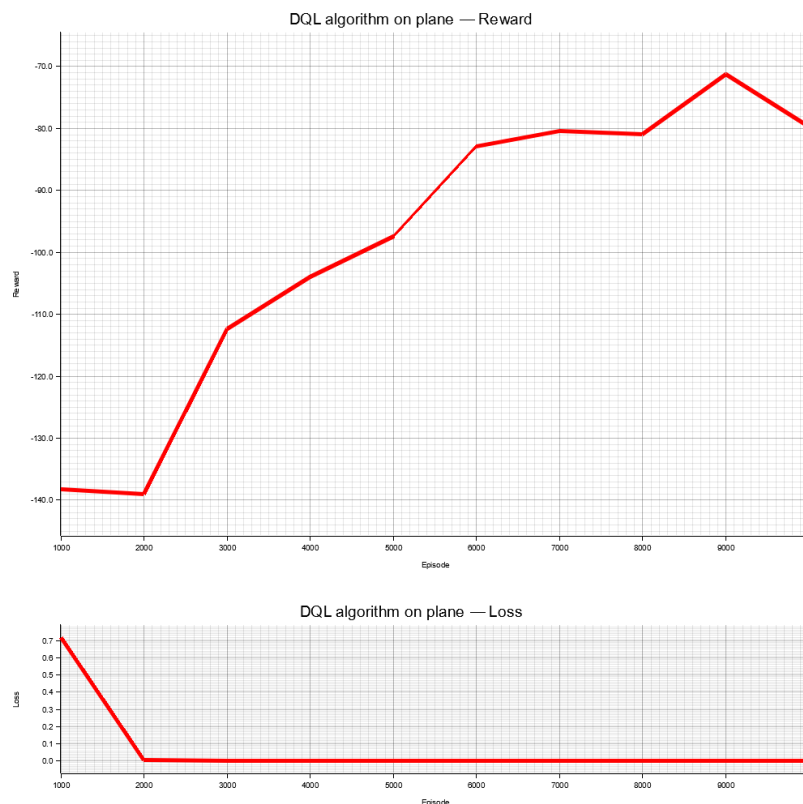


Figure 4.6: Mètriques del DQN amb millores

Observem que dona una millor solució que l'anterior, tot i que no arriba a solucionar el problema.

El DQL sembla l'evolució natural del tabular-Q-learning per entorns no finits, donat que utilitzen els mateixos fonaments (l'aprenentatge per diferències temporals), però aquesta última tècnica requereix moltes modificacions per tal de convertir-la en una tècnica robusta (altres millores comunes que s'apliquen són: Doble DQL, Priorització d'experiències en la memòria, Duel de DQL, ...). Per tant, donat que és en general un algorisme inestable sense aplicar-li moltes millores, deixaré aquí aquest algorisme i passaré a estudiar-ne d'altres.